



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Generic Classes and Methods

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Lecture Objectives

- ❖ Understand the problem without Generics
- ❖ Explain why type safety matters
- ❖ Recognize the need for Generic classes and methods
- ❖ Distinguish between:
- ❖ Compile-time vs Runtime errors

Introduction

- Generic methods and generic classes (and interfaces) enable you to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.
- Generics also provide compile-time type safety that allows you to catch invalid types at compile time.

Motivation for Generic Methods

In traditional Java programming, when a method needs to operate on multiple data types, developers typically use:

- ▶ Method Overloading: define multiple methods with the same logic but different parameter types

```
// OverloadedMethods.java  
// Printing array elements using overloaded methods.
```

```
public class OverloadedMethods {  
    public static void main(String[] args) {  
        // create arrays of Integer, Double and Character  
        Integer[] integerArray = { 1, 2, 3, 4, 5, 6 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };  
        Character[] characterArray = { 'H', 'E', 'L', 'L', 'O' };  
  
        System.out.println("Array integerArray contains:");  
        printArray(integerArray); // pass an Integer array  
        System.out.println("\nArray doubleArray contains:");  
        printArray(doubleArray); // pass a Double array  
        System.out.println("\nArray characterArray contains:");  
        printArray(characterArray); // pass a Character array  
    }  
}
```

```
// method printArray to print Integer array
public static void printArray(Integer[] inputArray) {
    // display array elements
    for (Integer element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
}
```



```
// method printArray to print Double array
public static void printArray(Double[] inputArray) {
    // display array elements
    for (Double element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
}
```

```
// method printArray to print Character array
public static void printArray(Character[] inputArray) {
    // display array elements
    for (Character element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
}
```

Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O

```
// OverloadedMethods.java  
// Printing array elements using overloaded methods.
```

```
public class OverloadedMethods {  
    public static void main(String[] args) {  
        // create arrays of Integer, Double and Character  
        Integer[] integerArray = { 1, 2, 3, 4, 5, 6 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };  
        Character[] characterArray = { 'H', 'E', 'L', 'L', 'O' };  
  
        System.out.println("Array integerArray contains:");  
        printArray(integerArray); // pass an Integer array  
        System.out.println("\nArray doubleArray contains:");  
        printArray(doubleArray); // pass a Double array  
        System.out.println("\nArray characterArray contains:");  
        printArray(characterArray); // pass a Character array  
    }  
}
```

Generic Methods: Implementation and Compile-Time Translation

- ★ If the operations performed by several overloaded methods are identical for each argument type, the overloaded methods can be more compactly and conveniently coded using a generic- method.
- ★ You can write a single generic method declaration that can be called with arguments of different types.
- ★ Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately.

Generic Methods: Implementation and Compile-Time Translation

- All generic method declarations have a type-parameter section delimited by angle brackets (< and >) that precedes the method's return type (< T > in this example).

- Each type-parameter section contains one or more type parameters (also called formal type parameters), separated by commas.
- A type parameter, also known as a type variable, is an identifier that specifies a generic type name.
- Can be used to declare the return type, parameter types and local variable types in a generic method, and act as placeholders for the types of the arguments passed to the generic method (actual type arguments).
- A generic method's body is declared like that of any other method.
- Type parameters can represent only reference types—not primitive types.

Generic Methods: Implementation and Compile-Time Translation

Printing array elements using generic method
PrintArray

```
// generic method printArray
public static <T> void printArray(T[] inputArray) {
    // display array elements
    for (T element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
} // end method printArray
```

Generic Methods: Implementation and Compile-Time Translation

```
public class GenericMethodTest {  
    public static void main(String[] args) {  
        // create arrays of Integer, Double and Character  
        Integer[] intArray = { 1, 2, 3, 4, 5 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };  
        Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };  
  
        System.out.println("Array integerArray contains:");  
        printArray(intArray); // pass an Integer array  
  
        System.out.println("\nArray doubleArray contains:");  
        printArray(doubleArray); // pass a Double array  
  
        System.out.println("\nArray characterArray contains:");  
        printArray(charArray); // pass a Character array  
    } // end main  
}
```

Generic Methods: Implementation and Compile-Time Translation

- ☾ When the compiler translates generic method `printArray` into Java bytecodes, it **removes** the **type-parameter section** and replaces the type parameters with actual types.
- ☾ This process is known as **erasure**.
- ☾ By default all generic types are replaced with type `Object`.

Generic Methods: Implementation and Compile-Time Translation

Before compilation

```
public static <T> void printArray(T[] array) {  
    for (T element : array) {  
        System.out.println(element);  
    }  
}
```

After compilation

```
public static void printArray(Object[] array) {  
    for (Object element : array) {  
        System.out.println(element);  
    }  
}
```

To be continued ... next week

Generic classes